

# DeepStyle

---

Photographer Search Style Engine

# Проблема и ценность проекта

---

## Проблема

Обычный поиск понимает, что изображено, но не учитывает стиль, настроение, свет и композицию.

## Кому нужно

Фотографы , дизайнеры, арт-директоры, контент-команды

## Ценность

DeerpStyle помогает находить изображения по содержанию, и по визуальному стилю, сокращая время поиска референсов.

# Dataset и Масштабирование

---

Реальный визуальный корпус

**24916**

изображений из Unsplash Lite

Оценка качества • Demo • Relevance

Синтетический корпус для масштаба

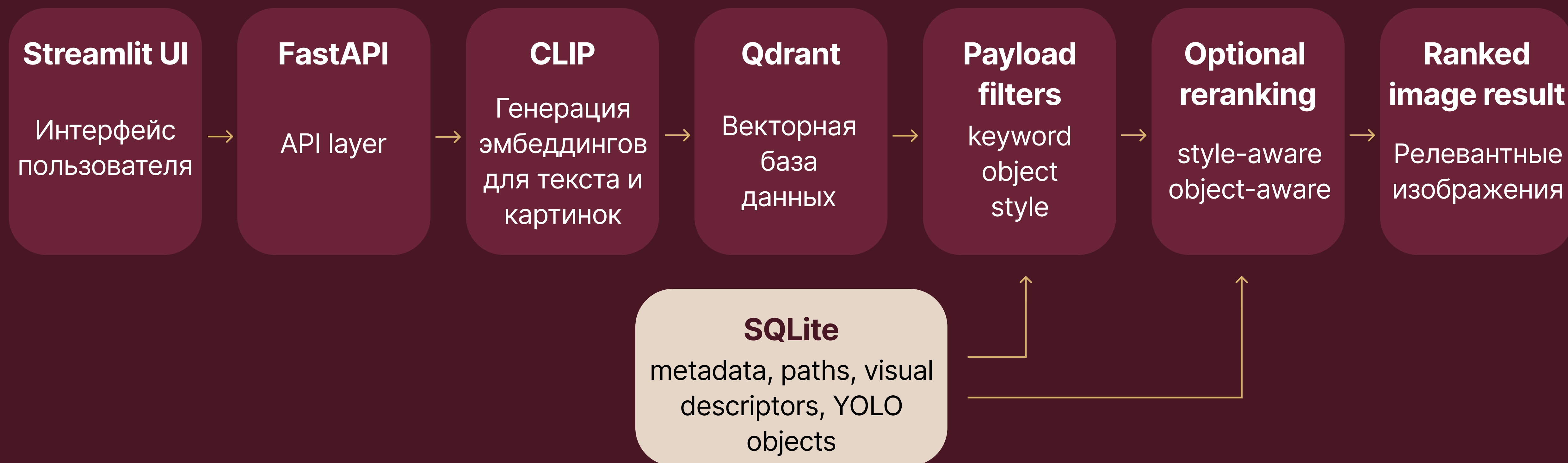
**500000**

сгенерированных CLIP-векторов

Latency • Memory • Scalability



# Общая архитектура системы



# Offline indexing pipeline



## Что хранится в базе (payload)

| Image_id                  | local file path       | CLIP vector (512d)            | keywords                     | detected_objects        | visual attributes                        | photo metadata                    |
|---------------------------|-----------------------|-------------------------------|------------------------------|-------------------------|------------------------------------------|-----------------------------------|
| идентификатор изображения | путь к файлу на диске | векторы признаков изображений | ключевые слова из метаданных | объекты, найденные YOLO | brightness, contrast, saturation, warmth | автор, дата, теги, источник и др. |

# Online search pipeline



## Режимы запроса во время поиска

**text-to-image**

текст → фото

**image-to-image**

фото → фото

**semantic search**

без фильтров

**filtered search**

payload-фильтры

**object reranking**

приоритет объектов

**style reranking**

цвет и стиль



# CLIP encoder

---



# FAISS Baseline

---

**CLIP  
embeddings**

Готовые 512d  
векторы



**L2  
normalization**

Векторы  
одинаковой  
длины



**FAISS  
IndexFlatIP**

exact local  
baseline



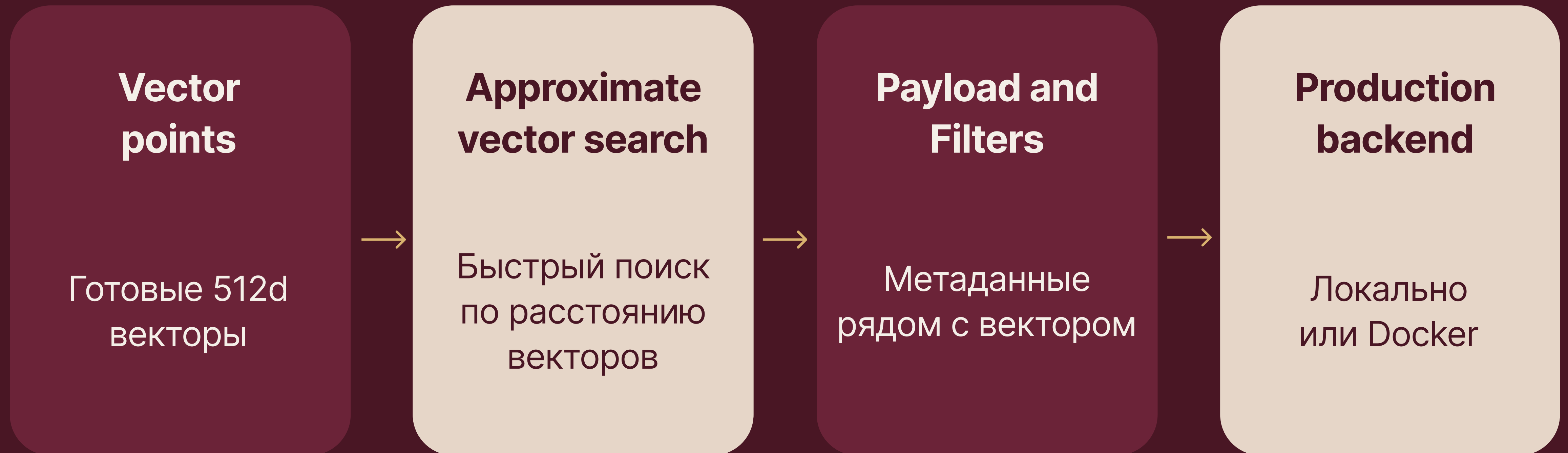
**Top-k  
results**

Semantic  
retrieval

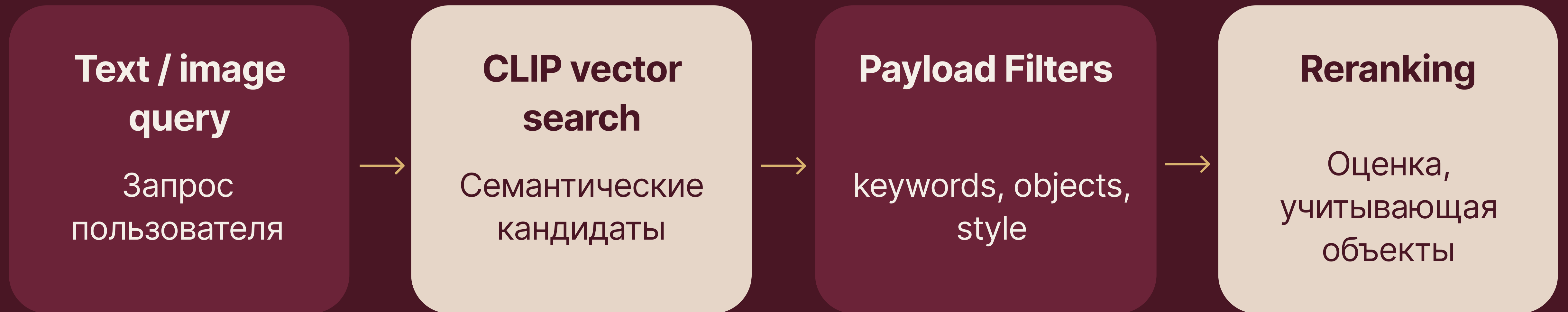


# Qdrant vector database

---



# Search и filtering logic



## Основные режимы

- qdrant\_semantic - CLIP vector search
- qdrant\_filtered - vector search and payload filters
- qdrant\_object\_rerank - кандидаты + object reranking

## Примеры фильтров

- keyword - beach, street, forest
- object - person, dog, car
- faiss\_style\_rerank - brightness, contrast, warmth

# Style descriptors и reranking

---



## Ограничения

Style reranking не гарантирует наличие нужного объекта

Полезен для тяжелых стилевых запросов, но не всегда лучше общего семантического ранжирования



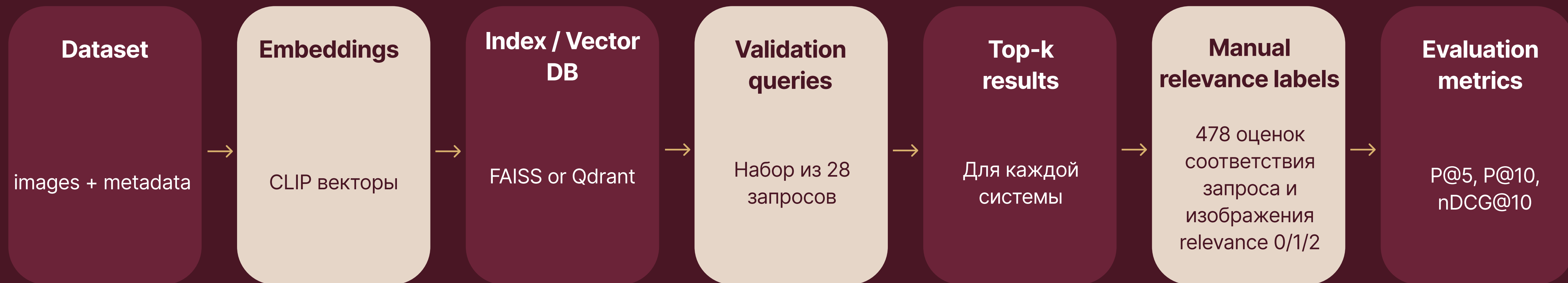
# YOLO object detection и reranking

---



# Экспериментальный pipeline

---



# Финальные метрики качества поиска

| System               | P@10   | nDCG@10 | MRR    |
|----------------------|--------|---------|--------|
| qdrant_filtered      | 0.9464 | 0.9481  | 0.9821 |
| qdrant_object_rerank | 0.9464 | 0.9393  | 0.9821 |
| faiss_style_rerank   | 0.8857 | 0.9155  | 0.9405 |
| faiss_baseline       | 0.9214 | 0.9134  | 0.9464 |
| qdrant_semantic      | 0.9214 | 0.9134  | 0.9464 |

**Лучший результат** выдает qdrant\_filtered: CLIP semantic search + payload ограничения



# Результаты по типам запросов

| Query type                 | Best system        | nDCG@10 |
|----------------------------|--------------------|---------|
| mixed_text (4 запроса)     | qdrant_filtered    | 0.936   |
| object_text (8 запросов)   | qdrant_filtered    | 0.989   |
| semantic_text (8 запросов) | faiss_style_rerank | 0.942   |
| style_text (8 запросов)    | qdrant_filtered    | 0.948   |

Filtering сильнее всего  
помогает object и mixed  
queries

Чистый семантический  
поиск уже хорошо  
работает через CLIP

Style reranking  
полезный, но не  
универсально лучший

# 500k scale benchmark

| Parameter       | Value                        |
|-----------------|------------------------------|
| collection size | 500k синтетических векторов  |
| vector size     | 512                          |
| distance        | cosine                       |
| deployment      | Qdrant server mode in Docker |

| Mode             | Avg latency (ms) | P95 latency (ms)  |
|------------------|------------------|-------------------|
| semantic         | 31.37            | 44.07             |
| keyword filtered | 202.52           | payload filtering |
| object filtered  | 355.85           | payload filtering |

Semantic mode показывает лучшую latency. Keyword и object filtered режимы работают медленнее из за payload filtering



# Оптимизация векторов

| Variant                      | Estimated 500k vector memory | Reduction | overlap@10 |
|------------------------------|------------------------------|-----------|------------|
| fp32 512d                    | 976.56 MB                    | 1.00x     | 1.0000     |
| float16 512d                 | 488.28 MB                    | 2.00x     | 0.9964     |
| int8 512d, python diagnostic | 246.05 MB                    | 3.97x     | 0.8750     |

float16 это лучший trade-off: 2x экономия памяти почти без изменения ранжирования  
int8 экономит больше памяти, но сильнее меняет ранжирование

# Docker и системные требования

## Окружение

Qdrant 1.18.2  
в Docker

500.000  
векторов

Метрика: Cosine

16 CPU, 16GB RAM

| Configuration                             | Recall@10 | p95 latency (ms) | Storage (GB) | Docker RAM (GB) |
|-------------------------------------------|-----------|------------------|--------------|-----------------|
| HNSW M16                                  | 1.0000    | 28.729           | 2.637        | 1.373           |
| HNSW M32                                  | 1.0000    | 30.529           | 2.579        | 1.299           |
| HNSW M64                                  | 1.0000    | 34.739           | 2.583        | 1.248           |
| Native INT8, native<br>Qdrant scalar INT8 | 0.8833    | 31.097           | 2.880        | 1.627           |

# Ограничения

---

**Unsplash keyword шумные**

автоматические теги могут быть неточными и неполными

**YOLOv8n использует фиксированные COCO classes**

например класс 'building' отсутствует, что ограничивает поиск по объектам

**500к синтетических векторов не являются реальными фотографиями**

используются только для scale benchmark и не показывают распределение реальных данных

**CPU YOLO inference медленный для полного корпуса**

полный пайплайн поиска по объектам рекомендует GPU для практического использования на масштабе



# DEMO

---